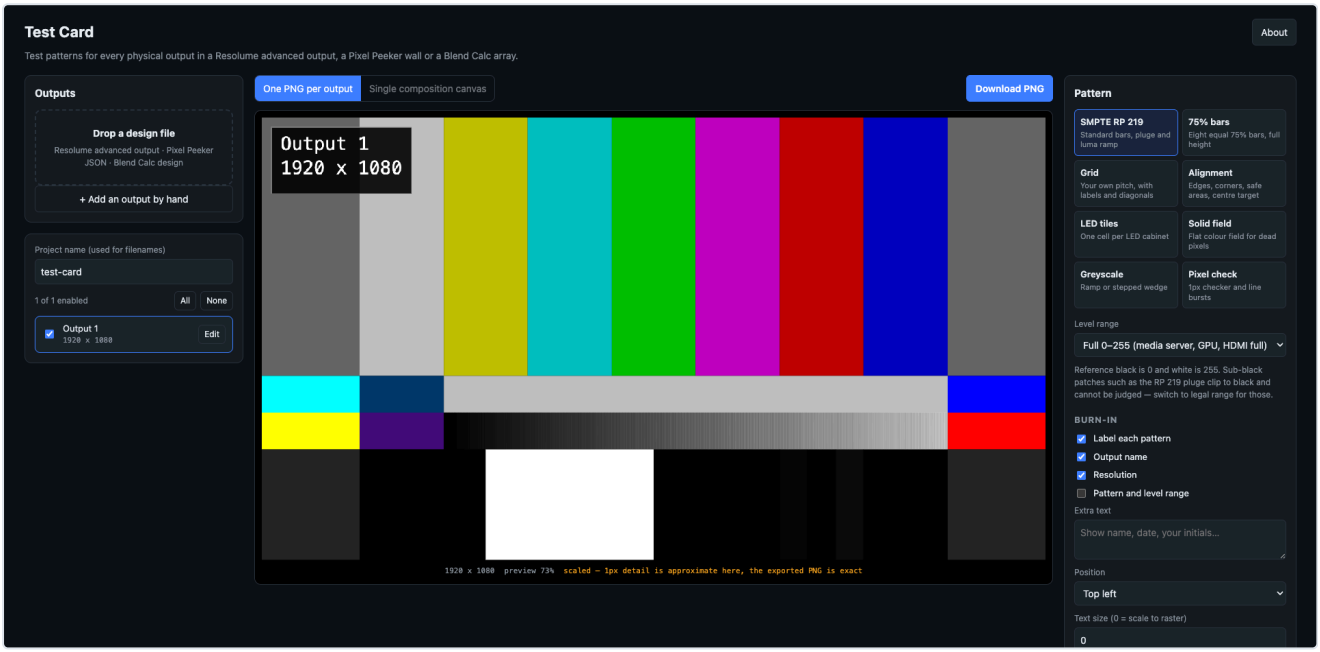
Test Card user guide

Test Card generates test patterns for every physical output in a show, from the file that already describes the outputs.



One 1920 × 1080 output added by hand, SMPTE RP 219 bars at full range with the output name and resolution burnt in. The preview is scaled; the exported PNG is pixel-exact.

Import a **Resolume advanced output**, a **Pixel Peeker** wall or a **Blend Calc** projector array, and get one PNG per physical output at that output's exact raster, or a single composition-sized image that proves the output map is what you think it is. Everything runs in the browser, and nothing is uploaded.

> **Before you rely on this:** the RP 219 pattern is checked against a reference **measured** from a real generator at four resolutions, rather than transcribed from prose descriptions of the standard, and its fifteen colours are computed through Rec. 709 and land exactly on those measured values, which is itself the proof the measurement is sound. The importers are tested against **real files**, not hand-written fixtures, and the whole path has been driven end to end in a browser.
>
> The MP4 loops are made in Chrome and read back by ffmpeg: 75% bars come back within one code value of the standard, and the loops go round with no seam. They've also been played in Resolume Arena 7.28.1, which opens them, gets the colour right and loops them cleanly, with one catch covered under **Moving patterns** below. A loop too big for the browser to encode whole comes out as tiles; decoded and stitched back together, an 8K one matches the picture drawn whole.
>
> **No image or loop it produces has been played out to a real projector, LED processor or display and measured.** The patterns are correct as pixels; what a screen does with them is something this can't tell you.
>
> This codebase was created with AI assistance, directed and reviewed by a human author.

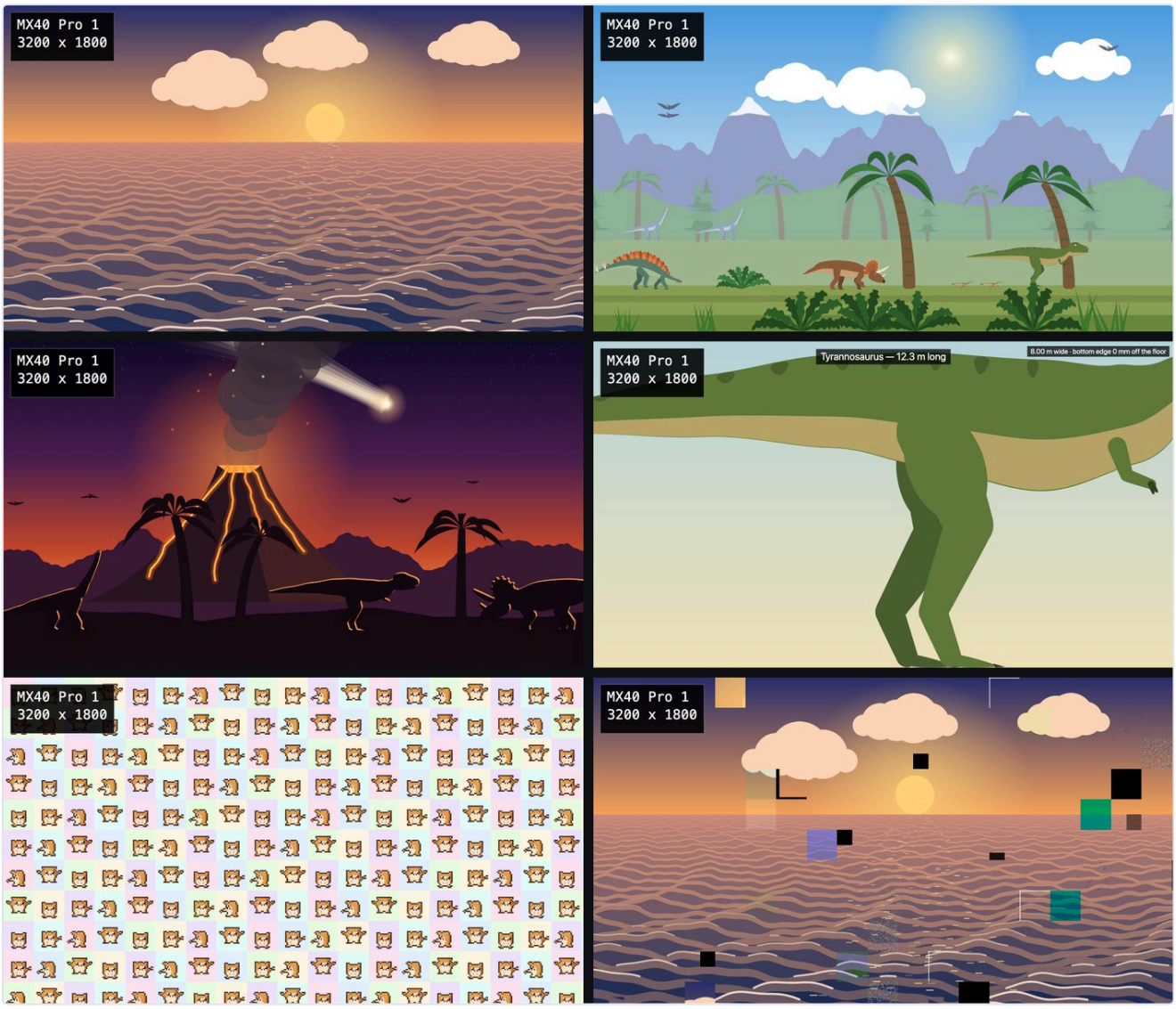
The patterns

Pattern	What it is for
SMPTE RP 219	Standard bars, +I/+Q, luma ramp and pluge.
75% bars	Eight equal full-height bars. Survives being squeezed onto a narrow LED strip where RP 219's four rows would be unreadable.
Grid	Your own pitch or division count, heavy lines every N, cell labels, diagonals, centre target.
Alignment	1px edge border, corner markers with pixel counts, safe areas, centre target and circles, 1px checker patches.
LED tiles	One cell per cabinet — either a uniform tile size you state, or the **real cabinet map** imported from Pixel Peeker, including irregular walls, chain order and per-port colouring.
Solid field	Flat colour at a chosen level, for dead pixels and uniformity.
Greyscale	Continuous ramp or stepped wedge, optionally split into R, G, B and neutral.
Pixel check	1px checkerboard and line bursts. **If it renders as flat grey on the real output, something in the chain is scaling.**

Every pattern can carry a burn-in (output name, resolution, pattern, plus your own text) and can draw the imported slice or port outlines on top.

Scale and novelty

The other sort of test pattern: the one that's up while the client walks in, or while you're explaining to someone what a dead module looks like.



Pattern	What it is for
Rulers	Tape measures along the edges, metric or imperial, from the pixel pitch. Each tick is within half a pitch of true, and the pattern says how much that is. The side rulers can read height off the floor.
Banana for scale	A life-size bunch of five 19 cm bananas, once or on every LED tile, with the wall's size in bananas.
Life-size dinosaurs	Seven dinosaurs at their usual quoted sizes, a 1.75 m person and a banana, standing on the floor in a line. **Pan** along it, or pick one under **Centre on**. Anything bigger than the wall is cut off by its edges.
Hamster dance	One dancing pixel-art hamster per LED tile, at a tempo you set. Moves.
Ocean waves	A generative sea rolling in, day, sunset or night. **Another** picks a different sea. Moves.
Dinosaur scroll	A side-scrolling Cretaceous with dinosaurs walking on the ground that scrolls under them. Moves.
Volcano and comet	An eruption, an incoming comet, and some worried silhouettes. Moves.
Broken wall	Another pattern with a share of its tiles broken in fifteen different ways, from a dead tile to a wrong RGB order. Tick **Label each fault** to name them on the wall.
Your image	Any picture, fitted, filled and cropped, stretched, 1:1 or tiled. It stays in this browser across a refresh and is never uploaded.

The per-tile patterns (bananas per tile, hamsters, the broken wall) use the same tiles as the **LED tiles** pattern: a uniform size you state, or the cabinets from a Pixel Peeker import.

Physical scale: give it a pitch

The rulers, the bananas and the dinosaurs are drawn at real size, so they need to know how big a pixel is.

- A **Pixel Peeker** import brings the wall's pitch with it.
- For anything else, open **Edit** on the output and type the **pixel pitch** in millimetres, or the image's **width in metres** and let it work the pitch out. For a projector, the width is the one you'll know.
- An output with neither is drawn at the **fallback pitch** in the pattern panel and says **pitch assumed** across the bottom. The ZIP manifest lists each output's pitch and marks the guesses.

Bottom edge above the floor puts the dinosaurs' feet on the real floor rather than on the bottom of the wall. A wall flown a metre up shows their knees, not their toes.

Carry on across outputs treats the outputs as windows onto one surface, placed by where they sit in the composition, so a ruler runs the whole wall and the sea has no seam between two projectors. It only applies to the scale and novelty patterns; bars and grids always belong to one output, and LED tiles stay with the output whose cabinets they are. An output stretched to a different shape in the composition isn't spanned and gets its own copy instead.

Moving patterns: MP4 loops

The moving patterns animate in the preview (**Pause** freezes it on the frame the PNG will use) and get a **Download MP4 loop** button. Set the length and frame rate under **Loop**.

- The loop goes round with no seam: everything in it does a whole number of cycles in the loop's exact length, which is a whole number of frames.
- It's H.264, encoded in your browser. That needs WebCodecs; I've only tried it in Chrome.
- Colour is BT.709, limited range, converted here and labelled in the file. Legal-range bars keep their sub-black and super-white.
- **Bitrate** at 0 picks a generous one for the raster. Set your own if the files are too big.
- Big rasters take a while: the export shows its progress and has a **Cancel**. It keeps going at the same speed behind another tab.
- A loop that would come to more than about 1 GB is refused before it starts, since the whole file has to fit in the browser tab's memory.
- **Too big for one video:** Chrome stops at 4096 × 2304 worth of pixels (9.4 megapixels, any shape), so a bigger raster comes out as a grid of **tiles**: 8K is four 3840 × 2160. A note above the preview says which outputs will be tiled and how, and the button says it's a ZIP. Each tile is its own seamless loop of the same length; the manifest and each file name (`…_tile-r1c2_3840x2160+3840+0.mp4`: row, column, size, then x and y) say where it goes. Place them at those positions and play them together. A bitrate you've set is shared between the tiles; auto is worked out per tile.

In Resolume. Arena 7.28.1 plays them at the right size, rate and length, with the right colour, and loops them without dropping or repeating a frame. The catch: an H.264 clip triggered from cold often plays for about half a second and then holds that picture until its first wrap. That's Arena's H.264 playback, not this file (ffmpeg's own H.264 encodes do the same; DXV doesn't). Before a show, convert the loop to **DXV** with Alley, which comes with Arena, or trigger the clip and let it run once round.

Two ways to render, and when each is the right one

One PNG per output, each at that output's native raster. One output downloads as a bare PNG; several download as a ZIP with a `manifest.txt` recording which file is which output, the level

range used, and any raster the app had to infer.

Single composition canvas: one image at composition size with every output's region drawn and labelled. Play it through Resolume and each physical output should light up showing its own name.

That second one is the fastest proof the advanced output map matches what you think it does, and you don't need any way of getting a file to each output individually, which on a show day is often the actual problem.

Level range: the app asks because a PNG cannot say

SMPTE bars are defined in Y′CbCr. A PNG is RGB, and a PNG can't record which convention it was written in. So the app asks, and writes the answer into the manifest.

- **Full 0–255** (default): media server, GPU output, HDMI/DP set to full range. **The −2% pluge patch clips to black and can't be judged.**
- **Legal 16–235**: studio swing, for an SDI chain. Sub-black and super-white survive, so the pluge means something. It looks washed out on a full-range display, and it's meant to.

Pick the one that matches the chain the file is going down, not the one that looks right on your laptop.

Imports, and what each file does not know

Resolume: one output per screen. The raster is read from the output device when it records one, and otherwise **inferred from slice bounds, which can come out too small.** Inferred rasters are badged in the UI and called out in the manifest, so correct them by hand.

Pixel Peeker: use **Export → JSON**, not *Save project*. The project file stores cabinets as library references in millimetres and can't be resolved to pixels here, so it's refused with a message naming the right export, rather than half-imported.

Blend Calc: needs the projector resolution (not in the file) and, for designs using a fit mode, the **solved** overlap (also not in the file, because Blend Calc works it out and never writes it back). Both are asked for. Its Resolume XML export needs neither and is the better file to bring.

You can also just add outputs by hand.

If something is wrong

Symptom	Cause
The pixel-check pattern is flat grey on the output	Something in the chain is scaling. That is exactly what the pattern is for.
An output's raster looks too small	It was inferred from slice bounds. It is badged; correct it by hand.
A Pixel Peeker file is refused	It is a project save, not the JSON export.
The pluge patch is invisible	Full-range levels clip it to black. Render at legal range for an SDI chain.
The bars look washed out	Legal-range content on a full-range display. Correct, not broken.
Blend Calc import asks for numbers	The projector resolution and the solved overlap are not in that file. Bring its Resolume XML instead.
"Pitch assumed" across the bottom	The output has no pixel pitch. Set it under **Edit**, or the life-size things are only as right as the fallback.
The dinosaurs' feet are missing	The wall's bottom edge is off the floor, and so are their feet. Set **Bottom edge above the floor** to 0 to see them.
No MP4 button	The pattern doesn't move. A broken wall moves if its picture does, or with flicker or data noise ticked.
"H.264 needs an even width and height"	4:2:0 video can't carry an odd raster. Export PNGs, or check the raster.
The MP4 comes as a ZIP of tiles	The raster is bigger than this browser will encode as one video. Place each tile where its name and the manifest say.
"The browser's H.264 encoder will not take..., whole or cut into as many as 64 tiles"	Even tiles didn't help, so this browser can't encode H.264 at all at this rate. Export PNGs.
An MP4 freezes after half a second in Resolume, then carries on at the wrap	Arena's H.264 playback from cold, not the file. Convert it to DXV with Alley, or let it run once round before the show.
"That loop would be about ... GB"	Too long, too big or too high a bitrate to build in a tab, tiles and all. Shorten the loop or lower the bitrate.
The image has gone	It's kept in this browser's storage. Another browser, a private window or cleared site data won't have it.